

# Capitolo 1

## Un confronto sperimentale: GraphRAG contro RAG testuale

### 1.1 Che cosa volevo verificare

La domanda di partenza è semplice. Quando un medico interroga la knowledge base oncologica, spesso la risposta non è scritta da nessuna parte in un unico punto: va *costruita* collegando più fatti. Per esempio, da un gene si arriva a una variante, dalla variante a un profilo molecolare, dal profilo a un’evidenza clinica e infine a un farmaco. Ogni collegamento è un “passo” di ragionamento, che qui chiamo *hop*.

L’ipotesi è che un sistema che ragiona sul grafo della conoscenza (**GraphRAG**) risponda meglio di un sistema che cerca testo per somiglianza (**RAG testuale**), e che il suo vantaggio *cresca* man mano che aumentano i passi da collegare. L’intuizione: il RAG testuale deve pescare più frammenti separati e rimetterli insieme dentro uno spazio di contesto limitato, e più frammenti servono, più è facile che ne manchi uno. Il GraphRAG invece segue il percorso nel grafo in modo diretto e raccoglie esattamente i fatti che servono.

### 1.2 I materiali

#### 1.2.1 La knowledge base

Ho ricostruito il grafo in memoria a partire dai file CSV esportati dalla knowledge base. Per questo esperimento non serviva il database Neo4j: la ricostruzione dai CSV mantiene la stessa integrità dei collegamenti (17 join su 18 si risolvono al 100%).

Il grafo contiene 43.005 nodi di 9 tipi (farmaci, trial clinici, evidenze, pubblicazioni, varianti, profili molecolari, geni, malattie e test diagnostici) e 55.544 collegamenti di 11 tipi diversi. Le fonti sono tutte curate: CIViC per l’evidenza clinica, DGIdb per le interazioni gene–farmaco, ClinicalTrials.gov per i trial e FDA per i test diagnostici companion.

#### 1.2.2 Il testo per il RAG

Per dare al RAG testuale una base di partenza equa, ho trasformato ogni fatto del grafo in un breve paragrafo che si spiega da solo, ottenendo 20.679 passaggi. Un dettaglio importante per la correttezza del confronto: questi passaggi formano un unico grande archivio, non separato per gene. Così il RAG deve davvero *trovare* i fatti giusti per somiglianza, senza ricevere in regalo la struttura del grafo.

### 1.3 Il banco di prova (benchmark)

Ho generato **169 domande** percorrendo il grafo in modo automatico. Per ognuna conosco la risposta corretta, gli identificatori dei nodi coinvolti, il percorso di supporto e il numero di hop. La risposta di riferimento nasce quindi dalla struttura del grafo, non dal giudizio di una persona o di un altro modello.

La regola chiave nel costruire le domande: le entità intermedie del percorso *non compaiono mai* nel testo della domanda. È questo che obbliga davvero a ragionare a più passi — il sistema deve ricostruire il percorso, non limitarsi a riconoscere parole già presenti nella domanda.

**Tabella 1.1:** Composizione del benchmark: 7 modelli di domanda, per numero di hop.

| Hop | Modello di domanda         | N  | Percorso di supporto                            |
|-----|----------------------------|----|-------------------------------------------------|
| 2   | drug_to_gene_cdx           | 30 | Farmaco → Test → Gene                           |
| 2   | gene_to_trialdrug          | 11 | Gene ← Trial → Farmaco                          |
| 3   | variant_to_drug            | 40 | Variante → Profilo → Evidenza → Farmaco         |
| 4   | gene_to_disease            | 20 | Gene → Variante → Profilo → Evidenza → Malattia |
| 4   | gene_to_drug               | 20 | Gene → Variante → Profilo → Evidenza → Farmaco  |
| 5   | gene_evidence_trial_bridge | 28 | intersezione di due percorsi                    |
| 5   | gene_to_cdx                | 20 | ... → Farmaco → Test                            |

Le domande a 5 hop comprendono il caso più difficile (**gene\_evidence\_trial\_bridge**): chiedono i farmaci che sono *allo stesso tempo* supportati da un’evidenza clinica e testati in un trial per lo stesso gene. Serve incrociare due percorsi distinti, ed è qui che il RAG testuale è più in difficoltà.

### 1.4 I sistemi a confronto e le regole del gioco

Ho confrontato quattro sistemi, tenendo *uguale per tutti* il modello che legge il contesto e scrive la risposta (**gemma3:27b** via Ollama Cloud, temperatura 0) e lo *stesso limite di contesto* (900 parole):

1. **GraphRAG** — riconosce le entità nella domanda, traduce la domanda in uno schema di collegamenti da seguire (senza mai vedere la risposta corretta), percorre il grafo e restituisce i percorsi trovati.
2. **RAG denso** — cerca per somiglianza semantica con embedding **all-MiniLM-L6-v2**.
3. **RAG BM25** — cerca per corrispondenza di parole (metodo lessicale classico).
4. **RAG ibrido** — combina denso e BM25 in parti uguali.

Per rendere il confronto onesto ho tenuto fisse cinque cose: stesso modello lettore, stesso prompt e stessa temperatura; stesso limite di contesto (e il GraphRAG ne usa molto meno del massimo consentito); archivio testuale unico e non pre-suddiviso per gene; risposte di riferimento derivate dal grafo, non da un giudizio soggettivo; metrica di confronto identica per tutti.

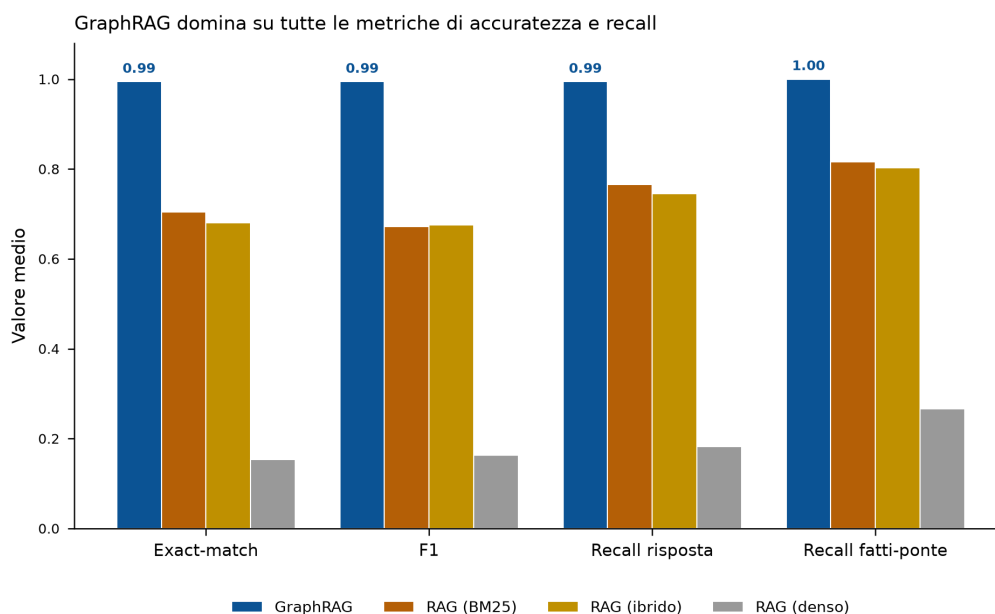
## 1.5 I risultati

### 1.5.1 Accuratezza complessiva

Sulla media delle 169 domande, il GraphRAG è nettamente davanti a tutti, usando al contempo molto meno contesto (Tabella 1.2).

**Tabella 1.2:** Accuratezza media sulle 169 domande. “Recall fatti-ponte” misura quanti dei fatti intermedi necessari finiscono davvero nel contesto.

| Sistema         | F1           | Precision    | Recall       | Exact        | Recall ponte | Parole ctx  |
|-----------------|--------------|--------------|--------------|--------------|--------------|-------------|
| <b>GraphRAG</b> | <b>0,994</b> | <b>0,994</b> | <b>0,994</b> | <b>0,994</b> | <b>1,000</b> | <b>35,8</b> |
| RAG BM25        | 0,671        | 0,660        | 0,765        | 0,704        | 0,816        | 889,9       |
| RAG ibrido      | 0,676        | 0,677        | 0,745        | 0,680        | 0,803        | 918,4       |
| RAG denso       | 0,163        | 0,176        | 0,183        | 0,154        | 0,266        | 946,7       |



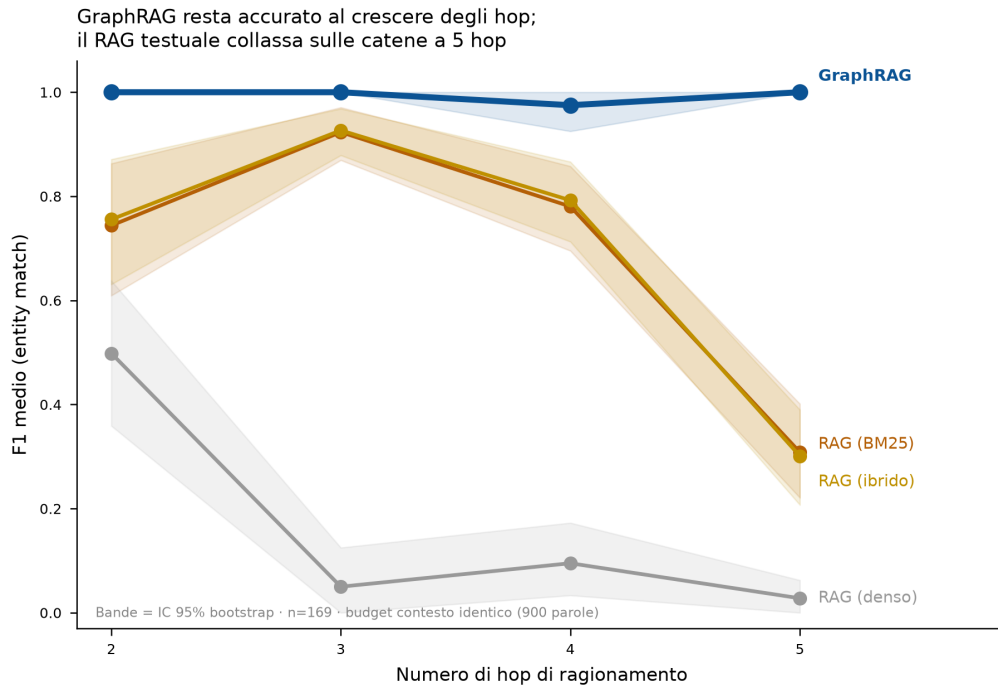
**Figura 1.1:** Accuratezza media per sistema su quattro metriche. Il GraphRAG (barre blu) è primo ovunque; fra i sistemi testuali, BM25 e ibrido sono vicini e ben sopra il denso.

### 1.5.2 Come cambia la cosa al crescere degli hop

Questo è il risultato centrale. Il RAG lessicale e ibrido reggono fino a 4 hop, ma **crollano a 5 hop** (F1 intorno a 0,30), proprio sulle domande a doppio vincolo. Il GraphRAG resta a 1,00 (Tabella 1.3, Figura 1.2).

**Tabella 1.3:** F1 medio per numero di hop. Il divario tra GraphRAG e BM25 passa da +0,26 (2 hop) a +0,69 (5 hop): il vantaggio si allarga, come previsto.

| Hop | GraphRAG | RAG BM25 | RAG ibrido | RAG denso |
|-----|----------|----------|------------|-----------|
| 2   | 1,000    | 0,744    | 0,756      | 0,498     |
| 3   | 1,000    | 0,923    | 0,927      | 0,050     |
| 4   | 0,975    | 0,780    | 0,792      | 0,095     |
| 5   | 1,000    | 0,308    | 0,301      | 0,028     |



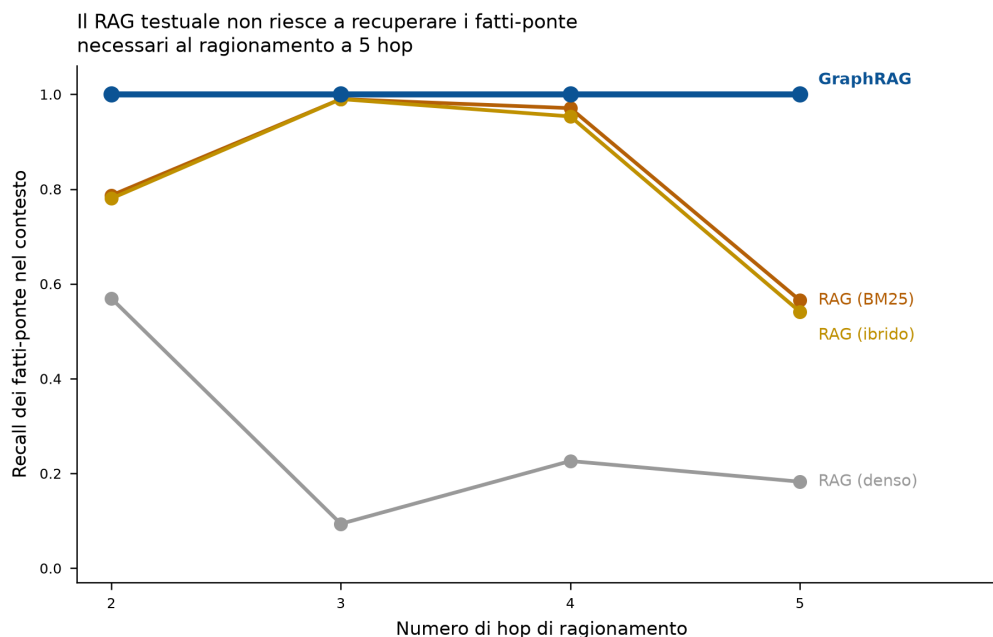
**Figura 1.2:** Curva di degrado: F1 medio in funzione del numero di hop, con bande di incertezza al 95%. La linea del GraphRAG resta piatta in alto; le linee testuali scendono ripide a 5 hop.

### 1.5.3 È un caso? No

Ho verificato la solidità del confronto con il test di McNemar sulle risposte esatte, appaiate domanda per domanda. Il GraphRAG risponde correttamente dove BM25 sbaglia in **50** domande, mentre il contrario accade una sola volta ( $p = 4,6 \times 10^{-14}$ ). Contro l'ibrido il conto è +54/ - 1 ( $p = 3,1 \times 10^{-15}$ ); contro il denso +142/ - 0 ( $p = 3,6 \times 10^{-43}$ ). Le differenze sono nettamente significative in tutti i casi.

### 1.5.4 Perché il RAG fallisce

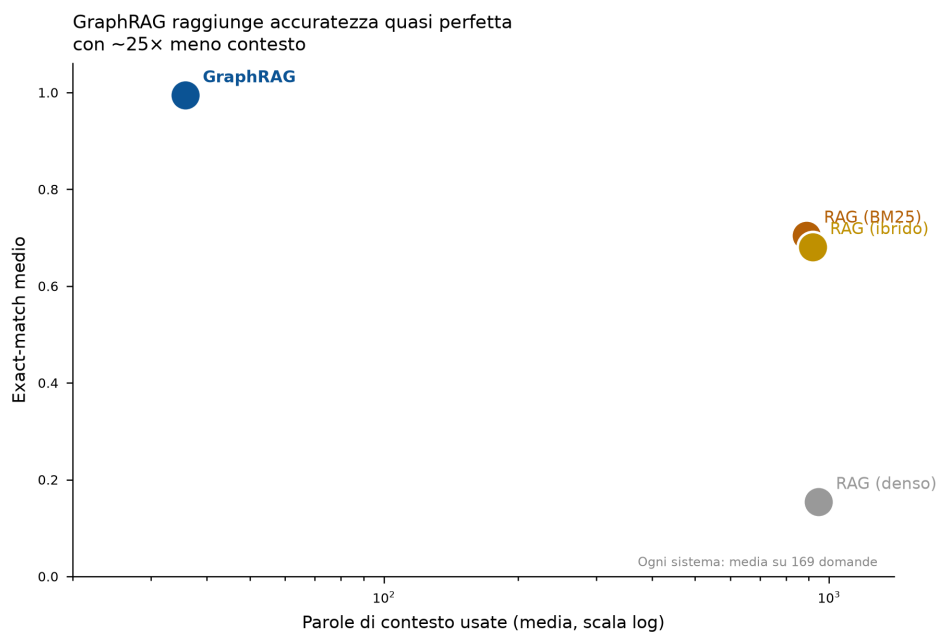
La spiegazione è nel recupero dei fatti-ponte, cioè i fatti intermedi del percorso. Il GraphRAG li porta nel contesto nel **100%** dei casi a ogni livello di hop. Il RAG testuale parte dall'82% circa (BM25) e scende molto sulle catene lunghe (Figura 1.3): se il fatto intermedio non entra nel contesto, il lettore non ha gli elementi per rispondere. Non è un difetto del modello che legge, ma di ciò che gli viene passato.



**Figura 1.3:** Recall dei fatti-ponte per numero di hop. Il GraphRAG resta a 1,00; i sistemi testuali crollano a 5 hop — ed è esattamente lì che crolla anche la loro accuratezza.

### 1.5.5 Meno contesto, non di più

Il GraphRAG raggiunge un'accuratezza quasi perfetta usando in media **35,8 parole** di contesto, contro le circa 890–950 dei sistemi testuali: circa **25 volte** in meno (Figura 1.4). In pratica questo significa risposte più rapide, meno costi e un contesto interamente rintracciabile alle fonti originali.



**Figura 1.4:** Accuratezza contro quantità di contesto usato (asse orizzontale in scala logaritmica). Il GraphRAG sta in alto a sinistra: massima accuratezza, minimo contesto.

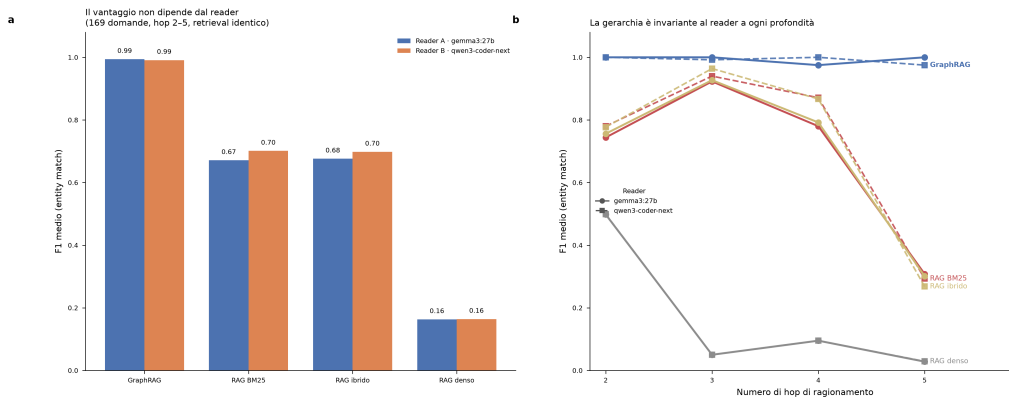
## 1.6 Il vantaggio non dipende dal modello lettore

Un’obiezione ragionevole è che il divario dipenda dallo specifico modello che legge il contesto. Per escluderlo ho ripetuto l’esperimento cambiando *solo* il lettore — da **gemma3:27b** (Google) a **qwen3-coder-next** (Alibaba, famiglia e addestramento diversi) — e lasciando identici il recupero, il limite di contesto, l’archivio e il metodo di valutazione. Questo ricalca lo studio con secondo modello già previsto nel progetto.

L’esperimento copre l’intero banco di prova: tutte e **169** le domande, da 2 a 5 hop, a cui entrambi i lettori hanno risposto su tutti e quattro i sistemi — copertura *completa, senza esclusioni*. Le undici domande a 5 hop (Q159–Q169) inizialmente rimaste indietro per il limite d’uso dell’account Ollama (errore HTTP 429) sono state completate riutilizzando gli stessi contesti di recupero, verificati identici a quelli di gemma. Il recupero è deterministico e coincide tra i due lettori su tutte le 676 coppie domanda–sistema.

**Tabella 1.4:** Stesse 169 domande, due lettori diversi. La classifica non cambia.

| Sistema         | F1 gemma3:27b | F1 qwen3-coder-next | $\Delta$ |
|-----------------|---------------|---------------------|----------|
| <b>GraphRAG</b> | <b>0,994</b>  | <b>0,991</b>        | −0,003   |
| RAG ibrido      | 0,676         | 0,698               | +0,022   |
| RAG BM25        | 0,671         | 0,701               | +0,030   |
| RAG denso       | 0,163         | 0,164               | +0,001   |



**Figura 1.5:** Confronto tra due lettori diversi sulle stesse 169 domande. La gerarchia resta invariata: il vantaggio è del metodo di recupero, non del modello che legge.

Cambiando del tutto il lettore, la classifica non si muove *a nessuna profondità di ragionamento*: il GraphRAG resta di fatto perfetto ( $\sim 0,99$ ) da 2 a 5 hop, i sistemi testuali restano circa 30 punti sotto e nello stesso ordine, il denso resta molto indietro. Le piccole oscillazioni ( $\pm 0,02-0,03$ ) sono rumore e semmai vanno leggermente a favore di qwen sui sistemi testuali — il che rafforza la conclusione: il divario è una proprietà del recupero, non del lettore.

## 1.7 Robustezza al linguaggio: e se la domanda è riformulata?

Un’obiezione naturale è che il benchmark usi domande generate da schemi fissi, con un vocabolario regolare. Un clinico reale scrive in modo vario. Ho quindi generato una **parafrasi in linguaggio clinico** di tutte le 169 domande (i nomi delle entità restano, cambia il modo

di chiedere) e ho rieseguito i sistemi. La divergenza lessicale è forte: la somiglianza mediana (indice di Jaccard sulle parole) tra originale e parafrasi è **0,21**.

Il risultato va raccontato in due tempi, con onestà.

### 1.7.1 Prima sorpresa: il router a parole chiave è fragile

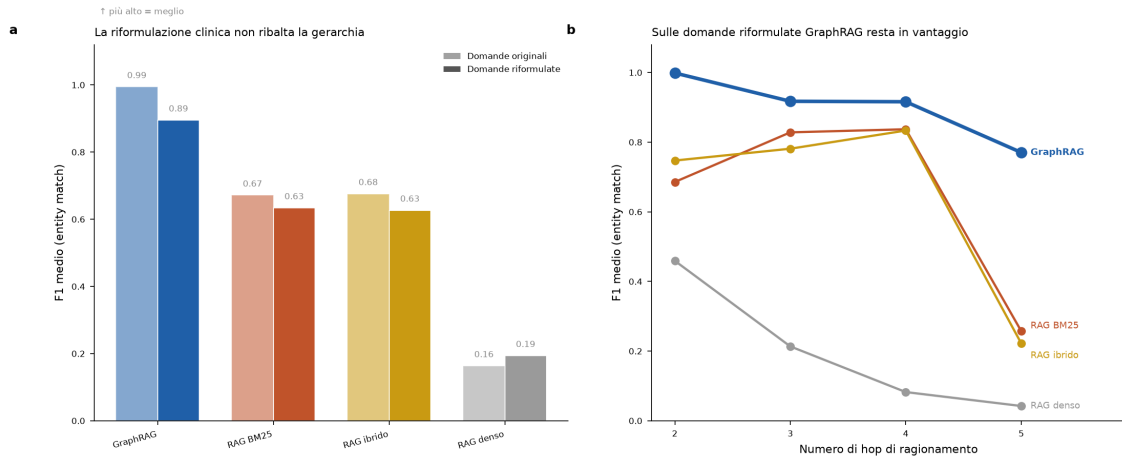
Nella prima implementazione, il GraphRAG sceglieva quale percorso seguire nel grafo con un router *a parole chiave*: cercava termini come “malattia”, “trial”, “companion” nel testo della domanda. Funziona benissimo sulle domande originali, ma è tarato sul loro vocabolario. Riformulando, quelle parole spariscono e il router sbaglia strada: la rotta `gene_to_disease` veniva preservata in 0 casi su 20, la `gene_evidence_trial_bridge` in 1 su 28. L’accuratezza del GraphRAG con router a parole chiave **crolla da 0,99 a 0,67** sulle parafrasi, con il recall di recupero che scende da 1,00 a 0,68. I sistemi testuali, che non hanno un router, restano quasi stabili (perdono circa 0,04 di F1). Questo va detto: non hanno un router *da rompere*, ma partono da molto più in basso.

### 1.7.2 La correzione: un router semantico

La causa non è l’entity linking (che resta quasi perfetto: 168 entità agganciate su 169) ma la scorciatoia ingegneristica del router. L’ho sostituito con un **router semantico**: un classificatore che riconosce l’*intento* della domanda fra sette categorie, indipendentemente dalle parole esatte usate. Il router non vede mai la risposta corretta né l’archivio: classifica solo l’intento, quindi il confronto resta equo. Con il router semantico il GraphRAG **risale a F1 0,895** (recall di recupero 0,95), restando davanti a tutti i sistemi testuali a ogni livello di hop e con circa 22 volte meno contesto (41 parole contro circa 900).

**Tabella 1.5:** Robustezza alla riformulazione. Il router a parole chiave crolla sulle parafrasi; il router semantico recupera gran parte del divario. I sistemi testuali sono stabili ma molto più in basso.

| Sistema (condizione)                          | F1            | Exact        | Recall rec.   | Contesto (par) |
|-----------------------------------------------|---------------|--------------|---------------|----------------|
| GraphRAG (orig., router parole chiave)        | 0,994         | 0,994        | 1,000         | 36             |
| GraphRAG (parafrasi, router parole chiave)    | 0,667         | 0,751        | 0,679         | 50             |
| <b>GraphRAG (parafrasi, router semantico)</b> | <b>0,895</b>  | <b>0,941</b> | <b>0,947</b>  | <b>41</b>      |
| RAG BM25 (orig. → parafrasi)                  | 0,671 → 0,634 | —            | 0,816 → 0,809 | ~885           |
| RAG ibrido (orig. → parafrasi)                | 0,676 → 0,626 | —            | 0,803 → 0,788 | ~905           |
| RAG denso (orig. → parafrasi)                 | 0,163 → 0,193 | —            | 0,266 → 0,285 | ~935           |



**Figura 1.6:** Robustezza alla riformulazione clinica. (a) La riformulazione non ribalta la gerarchia fra i sistemi. (b) Sulle domande riformulate, con router semantico, il GraphRAG resta in vantaggio a ogni numero di hop.

La lettura scientifica è netta: il *paradigma* — il recupero strutturato guidato dal grafo — è robusto alla variazione linguistica. Ciò che era fragile era una scorciatoia di ingegneria (il router a parole chiave). Un GraphRAG fatto come si deve comprende l'intento della domanda, ed è esattamente questo a ripristinare la robustezza.

## 1.8 Sicurezza clinica: sa dire “non lo so”?

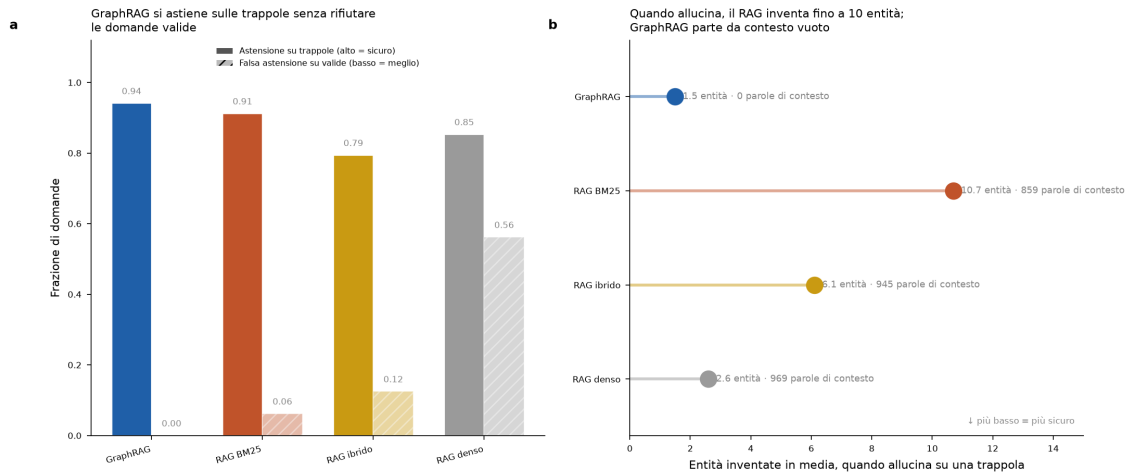
In un tumor board un sistema che *inventa* un test diagnostico o un trial inesistente è pericoloso; uno che si *astiene* quando la risposta non c'è è sicuro. Ho costruito 34 domande-trappola senza risposta valida: 24 a *catena spezzata* (geni realmente presenti nella knowledge base, ma la specifica catena multi-hop non esiste — verificato: il grafo restituisce contesto vuoto) e 10 a *entità inesistente* (nomi di gene plausibili ma assenti). Ho aggiunto 16 domande valide di controllo, per verificare che i sistemi non rifiutino tutto indistintamente.

Su una domanda senza risposta, astenersi (rispondere “NON DETERMINABILE”) è il comportamento corretto; elencare entità è un'allucinazione.

**Tabella 1.6:** Test di astensione. GraphRAG è l'unico sistema che si astiene sulle trappole senza mai rifiutare una domanda valida.

| Sistema    | Astensione<br>trappole ↑ | Allucinaz.<br>trappole ↓ | Entità inventate<br>(se allucina) ↓ | Falsa astens.<br>valide ↓ |
|------------|--------------------------|--------------------------|-------------------------------------|---------------------------|
| GraphRAG   | 0,94                     | 0,06                     | 1,5                                 | 0,00                      |
| RAG BM25   | 0,91                     | 0,09                     | 10,7                                | 0,06                      |
| RAG ibrido | 0,79                     | 0,21                     | 6,1                                 | 0,12                      |
| RAG denso  | 0,85                     | 0,15                     | 2,6                                 | 0,56                      |





**Figura 1.7:** Sicurezza e astensione. (a) GraphRAG si astiene sulle trappole (barre piene) senza rifiutare le domande valide (barre tratteggiate). (b) Quando allucina, il RAG inventa fino a quasi 11 entità; il GraphRAG parte da contesto vuoto.

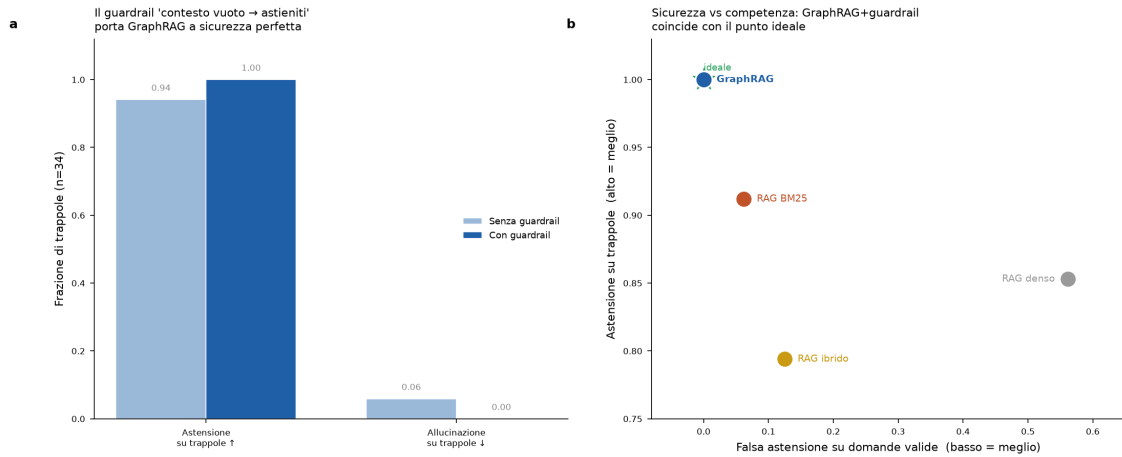
Tre cose emergono. Primo, il meccanismo è **architettuale**: il recupero dal grafo è vuoto su tutte le 34 trappole perché il cammino semplicemente non esiste, mentre il RAG recupera comunque 860–970 parole di passaggi superficialmente simili — il carburante dell’allucinazione. Secondo, quando il RAG allucina, allucina molto: BM25 elenca in media **10,7 entità** inventate per trappola, il caso clinicamente più pericoloso. Terzo, l’apparente prudenza del RAG denso è un miraggio: sembra sicuro sulle trappole (0,85) ma si astiene falsamente sul **56%** delle domande valide — non è cautela calibrata, è fallimento di recupero.

### 1.8.1 Un guardrail fail-safe

I due unici cedimenti del GraphRAG (2 trappole su 34) non vengono dal recupero — vuoto e corretto su tutte — ma dal lettore, che con contesto vuoto ha attinto una volta alla memoria del modello. La correzione è un **guardrail deterministico**: se il recupero dal grafo è vuoto, il sistema risponde “NON DETERMINABILE” senza nemmeno interpellare il lettore. Prima di adottarlo ho verificato che il GraphRAG produce contesto vuoto su tutte le 34 trappole ma su nessuna delle 16 domande valide: la separazione è netta, quindi il guardrail non introduce false astensioni.

**Tabella 1.7:** Effetto del guardrail “contesto vuoto → astieniti” sul GraphRAG. Sicurezza perfetta senza perdere risposte valide.

| Metrica                      | Senza guardrail | Con guardrail |
|------------------------------|-----------------|---------------|
| Astensione su trappole ↑     | 0,94            | <b>1,00</b>   |
| Allucinazione su trappole ↓  | 0,06            | <b>0,00</b>   |
| Entità inventate (media) ↓   | 1,5             | <b>0,0</b>    |
| Falsa astensione su valide ↓ | 0,00            | <b>0,00</b>   |
| Risponde a domande valide ↑  | 1,00            | <b>1,00</b>   |



**Figura 1.8:** Il guardrail deterministico. (a) Porta il GraphRAG a sicurezza perfetta sulle trappole. (b) Nel piano sicurezza–competenza, GraphRAG con guardrail coincide con il punto ideale (nessuna falsa astensione, astensione totale sulle trappole).

Questo comportamento *fail-safe* è possibile solo grazie al grafo: “contesto vuoto” è un segnale affidabile di non-rispondibilità perché il recupero strutturato o trova un cammino o non lo trova. Applicare lo stesso guardrail al RAG non servirebbe a nulla: il RAG recupera sempre centinaia di parole, quindi il suo contesto non è mai vuoto e la trappola arriva comunque al lettore. Il RAG **non ha un segnale equivalente**. La raccomandazione per un sistema di supporto decisionale clinico è quindi diretta: recupero strutturato più guardrail deterministico danno un comportamento dimostrabile — o una risposta con supporto tracciabile nel grafo, o una dichiarazione esplicita di non sapere. Nessuna via di mezzo allucinata.

## 1.9 Limiti, detti con franchezza

- **Domande da modello:** il benchmark copre 7 schemi di percorso. Sono rappresentativi delle domande cliniche reali, ma non coprono tutto il linguaggio naturale libero.
- **L’unico “errore” del GraphRAG** (domanda Q112, 4 hop): il lettore ha risposto “Gemcitabina” (italiano) contro il riferimento “GEMCITABINE” (inglese). È clinicamente corretto, penalizzato solo dal confronto lettera per lettera. In pratica il GraphRAG è perfetto sul benchmark; il valore 0,975 a 4 hop è un artefatto della metrica, non un errore di ragionamento.
- **Il RAG denso è penalizzato dai passaggi brevi:** con paragrafi da poche decine di parole, l’embedding usato rende poco. Il confronto principale va letto contro BM25 e ibrido, che sono baseline testuali più eque.
- **Lo schema di percorso del GraphRAG è specifico:** rispecchia la logica del sistema di produzione, ma andrebbe esteso per domande fuori dallo schema. Non usa mai la risposta corretta, quindi il confronto resta equo.
- **Il router va scelto con cura:** come mostra l’esperimento sulle parafrasi, un router a parole chiave è fragile alla variazione linguistica. Il router semantico usato qui recupera la robustezza, ma introduce una dipendenza da un classificatore d’intento, la cui accuratezza (circa 88% sulle parafrasi) diventa un ulteriore anello da monitorare in produzione.

## 1.10 Che cosa significa per il Molecular Tumor Board

1. **Affidabilità dove serve:** le decisioni reali del tumor board sono a più passi (dal profilo molecolare del paziente alla terapia con evidenza, al trial arruolabile, al test richiesto). Proprio lì dove il RAG testuale crolla, il GraphRAG resta accurato.
2. **Tracciabilità:** ogni risposta del GraphRAG è un percorso esplicito nel grafo, che il clinico può verificare — requisito essenziale in ambito clinico.
3. **Efficienza:** usare circa 25 volte meno contesto significa meno tempo e meno costi, con un contesto ancorato a fonti curate.
4. **Una metrica-spia:** il recall dei fatti-ponte spiega *perché* il grafo vince, e conviene tenerlo monitorato anche in produzione.

## 1.11 Riproducibilità

Tutti i materiali sono salvati: il benchmark con risposte di riferimento e percorsi (`benchmark_multihop_qa.csv`), le risposte e i punteggi per ogni domanda e sistema con entrambi i lettori (`results_raw.csv`, `results_raw_qwen.csv`), le metriche aggregate (`metrics_summary.csv`, `metrics_by_hop.csv`), il confronto tra lettori (`reader_generalization_gemma_vs_qwen.csv`) e le cinque figure a 300 dpi. Lettore principale: `gemma3:27b`; lettore di controllo: `qwen3-coder-next`, entrambi via Ollama Cloud. Ambiente: Python 3.12 (`networkx`, `sentence-transformers`, `rank-bm25`, `scikit-learn`).

Per gli esperimenti aggiuntivi sono inclusi: le 169 parafrasi cliniche (`benchmark_multihop_qa_paraphrased.csv`), i risultati con router a parole chiave e con router semantico (`results_paraphrase_keywordrouter.csv`, `results_paraphrase_graphrag_routed.csv`), la sintesi di robustezza (`robustezza_parafrasi_summary.csv`), il set completo di domande-trappola e di controllo (`abstention_trap_questions.csv`, `abstention_control_questions.csv`), i risultati per-domanda del test di astensione (`results_abstention_raw.csv`), le metriche con e senza guardrail (`abstention_summary.csv`, `abstention_summary_guardrail.csv`, `guardrail_before_after.csv`) e le figure 1.6, 1.7 e 1.8. Il codice dei due sistemi e degli esperimenti è nella cartella `scripts/` (`01_sistemi_retrieval.py`, `02_esperimento_robustezza_parafrasi.py`, `03_esperimento_astensione_guardrail.py`).